# Documentazione Tecnica di Progetto

Gestionale Locale per la Gestione del Magazzino

Gruppo 11

26 Maggio 2026

## Storico delle Versioni

| Versione | Data | Descrizione | Autore |
|---|---|---|---|
| v1.0 | 26/05/2026 | Strutturazione iniziale, analisi requisiti e allineamento UML | Gruppo 11 |

# Indice

# 1 Introduzione

## 1.1 Scopo del Progetto

Il software ha l'obiettivo di gestire le dinamiche logistiche interne di un'azienda, ottimizzando la gestione del magazzino, della movimentazione delle merci in entrata/uscita e della documentazione contabile associata.

## 1.2 Obiettivi e Ambito (Scope)

L'applicazione opera in un contesto puramente locale (offline). Questa scelta risponde a specifici vincoli ambientali e di sicurezza dell'infrastruttura ospitante, la quale potrebbe presentare zone prive di connettività internet stabile o politiche aziendali restrittive riguardo all'esposizione di dati sensibili all'esterno della rete aziendale. Il sistema gestisce l'intero ciclo di vita della merce: dalla registrazione dei prodotti a magazzino alla generazione dei documenti di trasporto (DDT) e della fatturazione.

## 1.3 Glossario

- **DDT**: Documento di Trasporto.
- **RBAC**: Role-Based Access Control (Gestione permessi basata sui ruoli).
- **MVC**: Model-View-Controller (Pattern architetturale).
- **DAO**: Data Access Object (Oggetto per l'accesso ai dati).
- **Stock-out**: Condizione di esaurimento delle scorte di un prodotto.

# 2 Analisi dei Requisiti

## 2.1 Attori del Sistema

- **Magazziniere**: Gestisce il flusso fisico delle merci (carico/scarico), effettua la scansione dei codici a barre, consulta le giacenze e genera i DDT per la merce in uscita.
- **Contabile**: Visiona l'elenco e il dettaglio degli ordini, gestisce le fatture, le note di credito e modifica i listini dei prezzi di acquisto e vendita.
- **Amministratore**: Supervisione globale del sistema, consultazione delle merci e dei prezzi, controllo delle fatture, gestione dei resi e generazione della reportistica avanzata dell'inventario.

## 2.2 Requisiti Funzionali (RF)

Tabella 1: Requisiti Funzionali

| ID | Descrizione | Priorità |
|---|---|---|

| | | |
|---|---|---|
| RF-01 | Login degli utenti tramite username e password basato su ruoli (RBAC). | Alta |
| RF-02 | Blocco automatico dell'account utente a seguito di 3 tentativi di login falliti. | Alta |
| RF-03 | Recupero della password utente tramite verifica di una domanda di sicurezza preimpostata. | Media |
| RF-04 | Visualizzazione, inserimento e modifica dei dati anagrafici dei prodotti e delle categorie. | Alta |
| RF-05 | Consultazione in tempo reale dell'elenco delle giacenze di magazzino. | Alta |
| RF-06 | Registrazione di un nuovo Ordine di Entrata e aggiornamento automatico incrementale delle giacenze. | Alta |
| RF-07 | Registrazione di un nuovo Ordine di Uscita e decremento automatico delle giacenze dei prodotti coinvolti. | Alta |
| RF-08 | Generazione automatica di un DDT per ogni Ordine di Uscita evaso. | Alta |
| RF-09 | Passaggio automatico dello stato dell'Ordine di Uscita da "Confermato" a "Evaso" alla creazione del DDT. | Alta |
| RF-10 | Notifica visiva immediata per i prodotti la cui giacenza scende sotto la soglia minima di sicurezza. | Media |
| RF-11 | Consultazione e gestione dei resi della merce con relativo ripristino delle quantità in giacenza. | Media |
| RF-12 | Visualizzazione e filtraggio degli ordini ricevuti (evasi e non evasi) da parte del contabile. | Alta |
| RF-13 | Generazione e conservazione di fatture di vendita e note di credito con numerazione progressiva annuale. | Alta |
| RF-14 | Modifica manuale dei listini dei prezzi di vendita e di acquisto dei prodotti. | Media |
| RF-15 | Integrazione software per l'acquisizione rapida dei codici a barre dei prodotti tramite lettore ottico. | Media |

## 2.3 Requisiti Non Funzionali (RNF)

Tabella 2: Requisiti Non Funzionali

| ID | Categoria | Descrizione |
|---|---|---|
| RNF-01 | Prestazioni | Il tempo di risposta alla scansione di un codice a barre deve essere inferiore a 0.5s per il 95% delle operazioni. |
| RNF-02 | Prestazioni | La generazione di un report annuale dell'inventario non deve richiedere più di 3s di elaborazione CPU. |
| RNF-03 | Portabilità | Funzionamento garantito in modalità offline su qualsiasi sistema operativo desktop moderno. |

| | | |
|---|---|---|
| RNF-04 | Sicurezza | Dati archiviati interamente in locale senza alcuna esposizione o transito all'esterno della rete aziendale. |
| RNF-05 | Archiviazione | Utilizzo di un database embedded SQLite, memorizzato in un singolo file binario locale. |
| RNF-06 | Usabilità | Interfaccia grafica (GUI) leggera, reattiva e ottimizzata per hardware a risorse limitate. |

## 2.4 Analisi dei Casi d'Uso

Il sistema è documentato tramite diagrammi dei casi d'uso che mappano le interazioni tra i tre attori principali e le macro-funzionalità del sistema. I flussi dettagliati delle operazioni principali sono descritti nei diagrammi di attività allegati nella cartella di progetto.




Figura 1: Diagramma dei Casi d'Uso Generale

# 3 Progettazione e Architettura

## 3.1 Scelte Architetturali

Il software adotta rigorosamente il pattern architetturale **Model-View-Controller (MVC)**. Questa scelta garantisce la netta separazione tra la logica di business e l'interfaccia utente, abbattendo il debito tecnico. Il motore di database scelto è **SQLite** per rispondere al vincolo di esecuzione locale e portabilità assoluta (zero configurazione iniziale, assenza di demoni di rete in background e facilità di backup tramite la copia di un singolo file binario).

## 3.2 Modello del Dominio

Il dominio dell'applicazione è incentrato sulle entità fondamentali per il tracciamento logistico, mappate nel file `Domain_model.puml`: `Utente`, `Prodotto`, `Categoria`, `Giacenza`, `OrdineEntrata`, `OrdineUscita` e `DDT`.




Figura 2: Modello del Dominio

## 3.3 Design Dettagliato (Diagramma delle Classi)

La struttura interna del codice segue rigorosamente lo schema delle classi definito in `Class_diagram.puml`. Per isolare la persistenza dei dati, viene adottato il pattern **DAO (Data Access Object)**. Le classi DAO incapsulano la totalità delle istruzioni SQL ed eseguono le query sul database SQLite, garantendo che lo strato dei Controller dipenda esclusivamente da oggetti di dominio e mai dal dialetto SQL sottostante.




Figura 3: Diagramma delle Classi

## 3.4 Comportamento del Sistema (Diagrammi di Sequenza)

I flussi di interazione dinamica tra i componenti della View, della logica di controllo (Service) e dello strato dei dati sono formalizzati tramite diagrammi di sequenza. Di seguito viene riportato, a titolo esemplificativo, il diagramma di sequenza relativo al processo di login e recupero delle credenziali (`UC1_login_sequence.puml`).




Figura 4: Diagramma di Sequenza: Login e Recupero Password (UC1)

# 4 Gestione del Progetto (Scrum)

## 4.1 Backlog

L'elenco ordinato e prioritizzato dei requisiti tecnici necessari al completamento del primo ciclo di sviluppo comprende:

1. Configurazione dell'architettura software di base, dell'ambiente di sviluppo locale e della connessione al file SQLite.

2. Definizione dello schema logico della base dati tramite script SQL (creazione delle tabelle `Utente`, `Prodotto`, `Categoria`, `Giacenza`, `OrdineEntrata`, `OrdineUscita`, `RigaOrdineEntrata`, `RigaOrdineUscita`, `DDT`).

3. Sviluppo delle interfacce grafiche e delle funzioni CRUD per la gestione dell'Anagrafica Prodotti e Categorie.

4. Implementazione del modulo di movimentazione (logica e maschere per la compilazione di `OrdineEntrata` e `OrdineUscita`).

5. Integrazione software per l'acquisizione dell'input da lettore di codici a barre tramite emulazione tastiera.

6. Sviluppo del sottosistema di controllo e notifica visiva per i prodotti sotto la soglia minima inseriti in `Giacenza`.

7. Progettazione ed implementazione del motore di reportistica ed esportazione dell'inventario in formato PDF e CSV.

8. Sviluppo del modulo di autenticazione locale e gestione dei permessi basata sui ruoli tramite `AuthService`.

## 4.2 User Stories

Le storie riflettono i bisogni concreti degli attori scritti secondo gli standard del framework Scrum:

- **US-01 (Scansione Barcode)**: «Come magazziniere, voglio poter scansionare il codice a barre di un prodotto in arrivo tramite il lettore ottico, per aggiornare la quantità in giacenza nel sistema in modo rapido, automatizzato e senza commettere errori manuali».

- **US-02 (Sicurezza Account)**: «Come amministratore del sistema, voglio che il software blocchi l'account dopo 3 tentativi falliti, per impedire ad utenti non autorizzati di indovinare le password d'accesso dei dipendenti».

## 4.3 Criteri di Accettazione

Vengono definiti i vincoli formali affinché la User Story **US-01** possa essere considerata completata ("Done"):

- **Criterio 1**: Il sistema deve riconoscere e decodificare correttamente e senza latenza i codici a barre conformi allo standard internazionale EAN-13.

- **Criterio 2**: Al momento della scansione riuscita all'interno della schermata di carico, i dati identificativi del prodotto devono comparire istantaneamente nei campi visivi e la relativa quantità nella tabella `Giacenza` deve essere incrementata automaticamente nel database.

- **Criterio 3**: Qualora il codice a barre scansionato non trovi corrispondenza all'interno della tabella `Prodotto`, il sistema deve inibire il salvataggio automatico, emettere un feedback acustico di errore ed aprire una finestra pop-up contestuale che consenta all'operatore di inserire il prodotto ex-novo in anagrafica o annullare l'operazione.

# 5 Strategia di Verifica e Validazione

## 5.1 Piano di Testing

Il piano prevede una suite di test di unità (Unit Testing) sistematici eseguiti sulla Business Logic tramite JUnit. I test sono focalizzati sulla verifica delle transazioni e sul rispetto dei vincoli di integrità del magazzino.

## 5.2 Casi di Test (Test Cases)

**TC-01**: Tentativo di scarico merce con quantità richiesta superiore alla giacenza fisica attuale.

- **Scenario**: Un prodotto registrato in anagrafica dispone di una quantità fisica in `Giacenza` pari a 5 unità. Viene inserito un `OrdineUscita` con una `RigaOrdineUscita` richiedente l'uscita di 6 unità del medesimo prodotto.
- **Input**: Invocazione della funzione di business `registraScarico()` con quantità = 6.
- **Risultato Atteso**: Il sistema deve intercettare preventivamente l'incoerenza prima di modificare i dati, bloccare l'esecuzione della transazione sul database SQLite tramite un'operazione di `ROLLBACK` e sollevare un'eccezione software personalizzata di tipo `GiacenzaInsufficienteException`. L'interfaccia utente cattura l'eccezione e mostra un messaggio di errore bloccante, impedendo che i dati assumano valori negativi incongruenti.